Assessing Accessibility Outcomes in AI-Assisted User Interface Development

Liza Babior Braxton Conley Trisha Gundavelli Rahat Bhatia

Abstract

More and more people build user interfaces with AI coding assistants such as Cursor, Claude Code, Codex or v0. These tools produce working interfaces fast, but we do not know how accessible these interfaces are, and how much this depends on the way the developer guides the assistant. In this project we will build the same set of web interfaces with a coding assistant under three levels of guidance. In the first level, the prompt does not mention accessibility. In the second level, the prompt asks for an accessible UI. In the third level, the assistant also gets a skill file with specific and strict accessibility guidelines. We will evaluate the results with automated accessibility checkers and a manual review of keyboard navigation and screen reader use. Our expected contribution is an empirical measure of how accessible AI-generated interfaces are by default, and how much each level of guidance improves them. Along with this evaluation we will study to what level of accessibility guidance is required before accessibility improvements plateau in AI-generated User Interfaces.

1. Problem Statement

Accessibility is still a big problem on the web. The WebAIM Million report checks the home pages of the top one million websites every year, and in 2026 it found detectable WCAG failures on 95.9% of them. Most of these errors are simple, such as low contrast text, missing alt text, and missing form labels.

At the same time, more and more web applications are being built with AI coding assistants. If these assistants produce inaccessible code by default, they can spread the same problems much faster than before. The W3C's Authoring Tool Accessibility Guidelines (ATAG 2.0) already say that tools which generate web content automatically should make it accessible, or at least prompt the author or check the result. AI coding assistants work like such tools, but we do not know if they meet this standard. On the other hand, they follow instructions, so a developer who

asks for accessibility, or gives the assistant clear rules, may get a much better result with little extra effort.

This project aims to investigate the following research questions:

- **RQ1:** How accessible are web interfaces generated by AI coding assistants without explicit accessibility guidance?
- **RQ2:** How much does accessibility improve when assistants receive general accessibility instructions versus structured skill files?
- **RQ3:** Do accessibility improvements show diminishing returns as the level of guidance increases?

2. Background and Related Work

Research in accessible LLM-generated code has shown that LLMs are capable of creating code that is more accessible than human-authored code. According to the paper Human or LLM? A Comparative Study on Accessible Code Generation Capability by Suh et al, LLM-generated code performed well on simple accessibility components such as contrast between the text color and background color and a text option to explain images. However, there were issues when creating more complicated accessibility components such as ARIA traits which is essentially about how different elements of certain roles need their role defined in their label.

Another related work is Can generative AI create accessible web code? A benchmark analysis of AI-generated HTML against accessibility standards by Doush and Kassem. The paper explores the differences in performance among different artificial intelligence models such as ChatGPT 4o and Claude 3.7 Sonnet to determine which model's generated code meets accessibility guidelines. Since focus was on web accessibility, the AI-generated code was used to create features such as tables, forms, images, and checkboxes. The study found that the models ChatGPT 4o and Claude 3.7 Sonnet had the best performance due to meeting more accessibility guidelines and requiring less further prompting.

While these papers present important findings, updated research is needed because large-language models have vastly improved within the last couple of years. We also aim to investigate how much guidance is needed to obtain sufficient accessibility, which seems to be novel in this scope of research.

4. Proposed Milestones

Milestone Report 1

- By the first milestone we plan to have completed several iterations of different User Interfaces using the 3 levels of prompts and begin evaluations of accessibility.

Milestone Report 2

- By the second milestone we plan to have complete automated evaluations and manual evaluations of accessibility and begin compiling results and writing the paper.

5. Scope, Limitations, and Contingencies

We will not run an active user study or try to recruit participants from a representative population, so we rely on automated checks and our own manual review instead. We focus only on web interfaces, because accessibility checkers and WCAG are most mature for the web. We also look only at accessibility, and do not judge if the interfaces look good or if the code is well written. Finally, we do not try to engineer the best possible prompt. We compare three fixed levels of guidance, and every assistant gets exactly the same briefs and instructions. This means we can see if the improvements plateau across our three levels, but not if some other prompt would do even better.

Automated checkers only find part of the accessibility problems, our manual review will cover more but we are students and not accessibility experts. We will use a checklist based on WCAG 2.2. AI coding assistants also change quickly and can give different results for the same prompt, so we will run each prompt more than once and record model version and dates. Results for the third level depend on the quality of the skill file we will use.

There are some non-negligible risks to our timeline that we must consider. The biggest one is likely going to be getting all the tooling setup for our automated evaluations. This is because we plan to use several different evaluators that excel in different areas and they may not all interface well with each model we use for development. Another risk to our timeline could be doing the manual evaluations and compiling all of the data which will likely take the largest chunk of time.